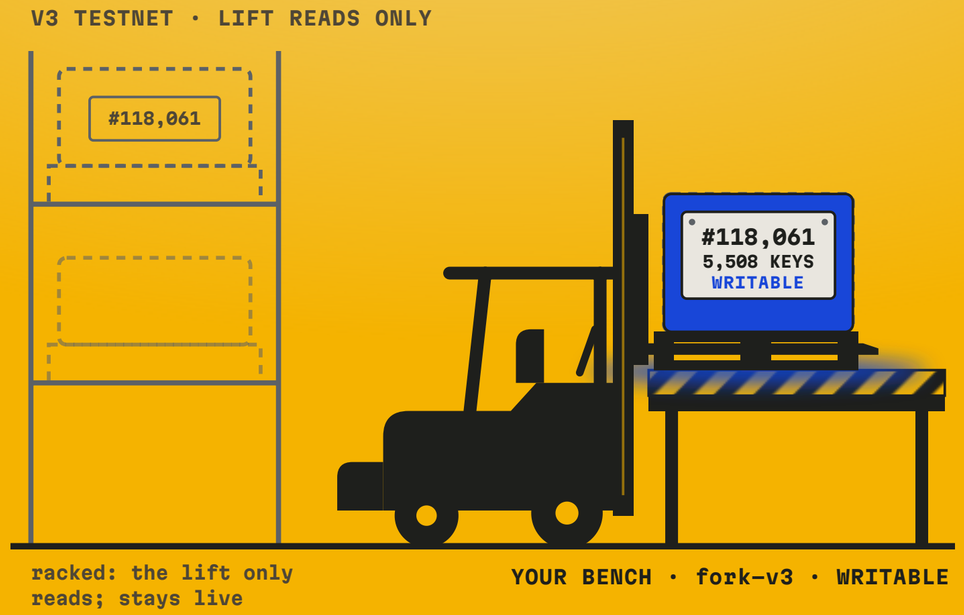
FORKLIFT

PORTALDOT HACKER HOUSE 2026 · NODE OPERATIONS & TOOLING

YOUR TRANSACTION RUNS TWICE. THE FIRST TIME IS A REHEARSAL.



KEYS LIFTED · #118,061	SAMPLED KEYS IDENTICAL	V3 SENDS, SAME TX HASH
5,508	200/200	3

```
$ forklift rehearse Balances.transfer_keep_alive … --send
chain    #118,621 118621-1 ok · finalized · fee …
         · tx 0x2247…c36f
compare  8/14 fields match · DIFFERS: …,
         inclusion_height, fee, …
$ forklift replay 118621-1
compare  14/14 fields match
```

V0.1.0 · MIT · PYTHON 3.11+ · AI-ASSISTED, SEE SLIDE 10

BEFORE THE LIFT · TWO PLACES TO TRY A TRANSACTION

THE DEV CHAIN IS EMPTY. THE TESTNET IS FINAL.

RACK · THE LIVE CHAIN

PORTALDOT V3 TESTNET

- Real state: accounts, balances, staking, contracts.
- Every transaction is final and pays its fee in tPOT.
- The fee multiplier changed between every pair of 20 consecutive blocks (**#118,206–#118,225**).
- No public V3 node binary or chain spec was found, so V3 cannot simply be run locally.

NOTHING CROSSES

BENCH · YOUR MACHINE

portaldot_dev --dev

- One validator, named Alice.
- None of the real chain's accounts, stakes or nominators.
- Writable, and free to break, but not the state your users are on.

A builder who wants to test one staking call, or one contract, against the state their users are actually on has no simple way to get that state onto a local node, and no way to check that what they got is faithful.

LIFT. FORK. VERIFY. REHEARSE. REPLAY.

BAY 01

LIFT

```
forklift up
--target v3 --at head
```

Reads every top-level storage key at one pinned block hash; **manifest_root** hashes every pair.

5,508 keys in 4.83 s
2 runs, same root

BAY 02

FORK

```
npx @acala-network/
chopsticks@1.5.2
```

Started by **up**: V3's own runtime wasm at the pinned block, on loopback, serving the pinned genesis and **:code**.

ready in 9.22 s
serves the pin: yes

BAY 03

VERIFY

```
forklift verify
--sample 200
```

Draws keys on the live chain and reads them on both; checks the whole key set and every lifted value.

exit 0 · 200/200
5,508 = 5,508

BAY 04

REHEARSE

```
forklift rehearse
<Pallet.call> --send
```

Signs once, applies the bytes on the fork (the prediction), sends the same bytes to V3, compares field by field.

3 V3 txs finalized
same tx hash

BAY 05

REPLAY

```
forklift replay
118621-1
```

Re-applies a V3 extrinsic's own bytes on a fork of its parent block and compares with the chain.

10/10 · 14/14
16/16, fees equal

CONTROL PATH

Mainnet state (spec 1002, read-only) spliced into the **portaldot_dev** 2.0.0 node (community tarball), labelled **fork-1002-fast**, not V3: fund an account, act as a real whale and nominator, and a 169-byte Fork Receipt contract.

LIVE V3 PROOF · SIGNER 5EnCf9Pa…ejvg · 2026-10-20 · ALL FINALIZED

PREDICTED ON A FORK. THEN SENT TO V3.

System.remark

a remark, no recipient

118555-1

tx 0x5b4b4d48…1a65777d

MATCHED THE PREDICTION

- ✓ same tx hash, success
- ✓ same event sequence

DIFFERED · 6 OF 10 FIELDS · EXIT 7

- fee 8,269 planck lower: in the fee, its 3 fee events and the sender's delta
- included in **#118,555**, predicted **#118,546**

REPLAY ON PARENT #118,554

10/10 fee equal

Balances.transfer_keep_alive

0.2 tPOT to a new account

118621-1

tx 0x2247c66b…73fdc36f

MATCHED THE PREDICTION

- ✓ same tx hash, success, event sequence
- ✓ recipient +0.2 tPOT, exact

DIFFERED · 6 OF 14 FIELDS · EXIT 7

- fee 3,660 planck lower: in the fee, its 3 fee events and the sender's delta
- included in **#118,621**, predicted **#118,618**

REPLAY ON PARENT #118,620

14/14 fee equal

Utility.batch

remark + 0.1 tPOT transfer

118669-1

tx 0xd371c50a…a3a3ef6a

MATCHED THE PREDICTION

- ✓ same tx hash, success, event sequence
- ✓ recipient +0.1 tPOT, exact

DIFFERED · 6 OF 16 FIELDS · EXIT 7

- fee 5,390 planck lower: in the fee, its 3 fee events and the sender's delta
- included in **#118,669**, predicted **#118,665**

REPLAY ON PARENT #118,668

16/16 fee equal

Signed once: the bytes ran on the V3 fork first, then went to the live testnet. The fee events differ by thousands of planck; each landed 3 to 9 blocks after the predicted height, and V3's fee multiplier moves every block. **rehearse** reports it (exit 7) and never adjusts a prediction; a replay on the parent block matches field for field.

IS THE FORK FAITHFUL? CHECKED AGAINST THE CHAIN, NOT ASSUMED

THE FORK ANSWERS TO THE CHAIN.

V3 TESTNET · VERIFY AT #118,061 · EXIT 0

200/200

draws on the live chain (198 distinct keys), all identical on the fork (seed **3145772797887710471**)

5,508=5,508

key set on the fork equals the lift

5,497

of 5,508 lifted values identical; the other 11 are child-trie roots the fork reads as null: counted and listed, not hidden

REPLAYS ON PARENT BLOCKS · FEES EQUAL TO THE PLANCK

10/10 · 14/14 · 16/16

The lift was run twice from fresh processes: same **manifest_root 0xfe4962de…48c430a4**. A sample is not a proof of every key, so V3 verify also compares every lifted value.

CONTROL · FORK-1002-FAST · NOT V3

MAINNET STATE, SPEC 1002, PIN #2,725,953

16,324=16,324

keys lifted = keys in an independent full listing of the same block, outside the MMR range

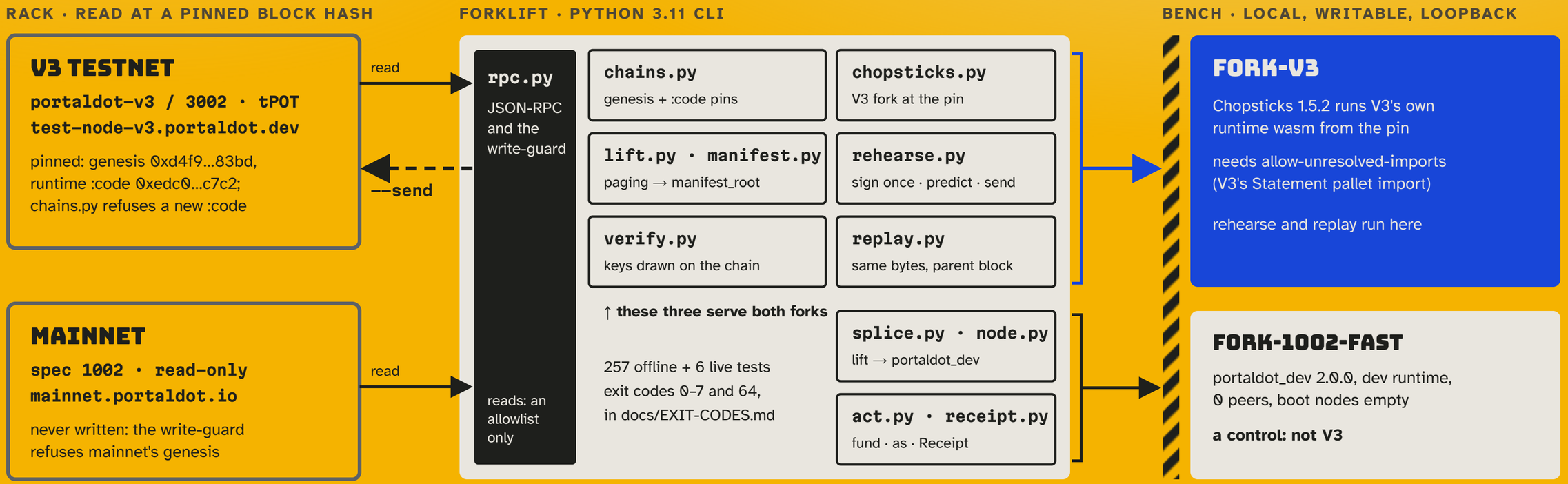
0 missing · 0 extra

full listing: 5,468,224 keys, 5,470 pages, 645.6 s

exit 0

verify --sample 200: 120 identical, 80 dev-kept (consensus and System keys the dev node keeps)

RACK → FORKLIFT → BENCH



`rehearse --send: author_submitExtrinsic only, V3 only (dashed)`

The one write path to a real chain. Every other write goes to a loopback fork.

Every run writes **`evidence/*.json`**; **`scripts/proof.py`** generates **`docs/PROOF.md`** from it.

PREDICT. SEND. COMPARE.

ONE SIGNATURE, BOTH SIDES

The extrinsic is signed once, so the tx hash on the fork is the tx hash on V3. The prediction and the chain's record are about the same bytes.

WRONG IS AN EXIT CODE

When the chain differs, **`rehearse`** exits 7 and lists each field. A predicted failure exits 5 and sends nothing (an unfunded signer: the fork's pool rejected it).

FIDELITY YOU CAN RE-RUN

`forklift replay <block>-<index>` re-applies any V3 extrinsic on a fork of its parent block and compares it with the chain.

PINNED BY WHAT THE CHAIN SERVES

Genesis plus runtime **`:code`** hash, not a URL. Before predicting, the fork is advanced toward the chain's best block, and the fee multiplier is re-synced if it differs.

	V3'S REAL STATE	OUTCOME KNOWN BEFORE SENDING	CHECKED AGAINST THE CHAIN
A dev chain	no: Alice only	yes, on empty state	no
Send to V3 directly	yes	no: the tx is final	after the fact
Forklift	**#118,061, 5,508 keys**	**predicted, then sent: same tx hash**	**verify exit 0; replays exact**

BUILT ON PORTALDOT'S OWN NODES.

USES

THE NODE, NOT A MOCK

- The historical-state RPC at a block hash: **`state_getKeysPaged`**, **`state_queryStorageAt`**.
- The **`portaldot_dev`** 2.0.0 node (community tarball), booted with zero peers.
- V3's own runtime wasm (spec 3002) on the fork.
- Both chains: the V3 testnet and mainnet state (spec 1002).

GIVES BACK · DOCS/DX-REPORT.MD

12

FRICTION POINTS, EACH WITH HOW TO SEE IT AGAIN

- The macOS **`portaldot_dev`** binary is x86_64 only (runs under Rosetta).
- **`--ws-port`** is the WebSocket port, **`--rpc-port`** the HTTP port.
- Chopsticks 1.5.2 cannot fork V3 with default settings.
- V3's fee multiplier moves every block.

OPEN SOURCE

MIT, EVIDENCE-FIRST

- 257 offline + 6 live tests.
- 83 commits to build **`44b7edc`**, dated 2026-10-14 to 2026-10-20.
- **`docs/PROOF.md`** is generated by **`scripts/proof.py`** from **`evidence/*.json`**.
- Documented exit codes and **`--json`** output, so a CI job can gate on a fork.

BUSINESS POTENTIAL

WHO LIFTS, AND HOW IT PAYS.

BUILDERS

Test one staking call or one contract against the state users are actually on, before spending tPOT on a transaction that is final.

NODE OPERATORS

Rehearse an operational transaction on a fork of the chain they run, and see the events, fee and balance deltas before sending.

WALLETS & EXCHANGES

Predict the outcome of a transaction on a pinned copy of the chain inside CI, and fail the job when the chain would differ.

TODAY · BUILT

A free MIT command-line tool, installed from the repository.

NEXT · ROADMAP

A PyPI package, a CI action and a hosted fork.

AFTER THE EVENT · PLAN

Paid hosted per-team forks and CI fork-as-a-service for wallets and exchanges; a shared fork, re-pinned at each runtime upgrade, offered to the Portaldot Foundation to host.

Honest limit: no revenue and no external users yet. The business case is a funded public good plus CI minutes for integrators.

STATUS AT THE CHECK-IN · BUILD 44b7edc

WHAT IS DONE, WHAT IS LEFT.

DONE AND MEASURED

- ✓ V3 profile pinned; lift, fork and verify at **#118,061**, exit 0.
- ✓ **rehearse --send**: 3 V3 transactions, all finalized.
- ✓ **replay**: all 3 equal to the chain, fees to the planck.
- ✓ Mainnet control: lift with 0 missing / 0 extra, fork, verify, fund, act as real accounts, Fork Receipt.
- ✓ 257 offline + 6 live tests · 83 commits.

BEFORE THE FINAL

- The Fork Receipt on V3, as an ink! 5 / Revive contract.
- **--runtime exact**, a runtime-difference report, replay of a mainnet extrinsic.
- A mainnet (1002) → V3 report: which keys carry over.
- Benchmarks with p50/p95; a hosted fork, a PyPI package, a CI action.

LANDING · SCAN THE CODE
forklift.edycu.dev

REPOSITORY
github.com/edycutjong/portaldot-hackathon-2026-forklift-edycu

PROOF TABLES
docs/PROOF.md

ONE V3 TX ON NODE-CONSOLE
118621-1

AI assistance: Claude Code drafted most of the code, tests, README and this deck under the author's direction. The architecture, protocol choices and on-chain design were decided by the author, who reviews the code and can explain how the lift, the splice, **verify** and the Receipt work. Every figure here comes from **docs/PROOF.md**, **evidence/**, **README.md** or the git log at build **44b7edc**.